Building and Trusting a Model

Unit 4: Many Predictors, Overfitting, and Honest Evaluation

Stat 220 | Statistics for Data Science

Where we left off

- Unit 3 fit a line and learned to read a slope, including what happens to it when another predictor joins the model.
- Now we build models with *many* predictors and confront the question that decides whether any of it is real: **how well does this do on data it has never seen?**
- Three days: putting a model together, checking it honestly, and deciding how much flexibility the data can actually support.

Principle: the tension, stated plainly

A model that fits your data perfectly has told you nothing except what your data already said.

The tension in every model you build

- A model has one real job: do well on data it **hasn't seen**.
- But we only ever fit it on data it *has* seen. So a model can score well for two very different reasons: it learned the real signal, or it memorized the noise.
- Almost everything in this module is about telling those two apart.

In practice: the shape of this module

"Your R^2 jumped from 0.6 to 0.95 when you added more variables. Is it a better model?"

What this unit gives you

Things to know

- multiple regression, and what "holding the others fixed" really means
- interactions and transformations
- multicollinearity, and what it does to coefficients
- overfitting, and why training error is not evidence
- cross-validation done correctly, including where leakage sneaks in
- regularization and trees: more flexibility, honestly measured

Mistakes to stop making

- judging a model by its fit to the training data
- chasing R^2
- preprocessing or selecting features outside the folds
- tuning on the test set and then reporting that score
- quoting p-values from a model built by variable search
- reading feature importance as a causal effect

First, let's "verify" on the same 10 people

- We feed each of our 10 volunteers' heights back into the line and compare the predicted shoe size to their real one.
- The predictions are great. The errors are small. R^2 looks high.
- So the model is excellent. Are we done?

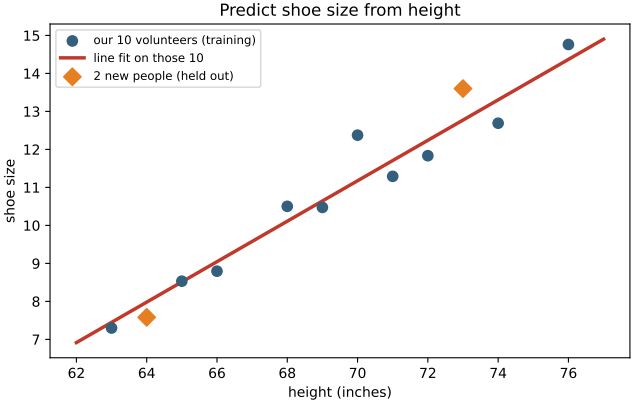
First, let's "verify" on the same 10 people

- We feed each of our 10 volunteers' heights back into the line and compare the predicted shoe size to their real one.
- The predictions are great. The errors are small. R^2 looks high.
- So the model is excellent. Are we done?

Trap: that was grading your own homework

Of course it fits those 10. We *built* the line on them. Checking a model on its own training data tells you almost nothing about how it will do on someone new.

The honest test: people we held out



Now bring up **2 new volunteers** (the orange diamonds) who were *not* used to fit the line. Predict their shoe size, then check.

The error on *those* people is the number that counts. That is the whole idea of a **train/test split**: fit on some data, judge on data the model never saw.

From one predictor to many

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_k x_k$$

- Each coefficient is the effect of its predictor **holding all the others fixed**, the idea we leaned on for confounding in Part 2.
- More predictors can mean a sharper model, or a tangled one where the coefficients fight each other. Which one you get depends on how the predictors relate.

Principle: a model is a team of predictors

The coefficients are not judged one at a time in isolation. Each is reported *given* the others in the room. Change the lineup and the coefficients change.

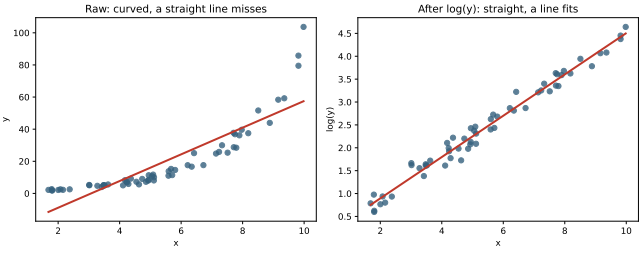
Interactions: when the effect of x depends on something else

A plain model forces one slope for everyone. An **interaction** term lets the slope itself change by group.

- "The effect of price depends on whether it's a weekend."
- Lines are no longer parallel.

Add them when you have a real reason to think the effect differs, not just to raise the fit.

Transformations: when straight lines don't fit



Many real relationships curve. A **log** (or a squared term) can straighten them so a linear model fits.

The cost is interpretation: after a log, a coefficient is read as a *percent* change, not an absolute one. The model gets simpler and the sentence gets more careful.

Your turn #1

Your turn: which model, and why

You're modeling house price from square footage, and the scatter clearly **curves upward**: big houses cost more than a straight line predicts.

Turn to a neighbor: would you add a **squared term**, take a **log**, or leave it straight? Practice explaining the trade-off in one sentence.

Your turn #1: answer

- Either a squared term or a log of price can capture the curve, and both fit better than a straight line here.
- The trade-off is **interpretation**: a log lets you say "each extra 100 sqft is about $x\%$ more," which is often the cleaner story for prices.
- Leaving it straight is the one clearly wrong choice: the model would under-predict big houses and over-predict small ones in a systematic way.

Principle: fit the shape, then mind the sentence

Pick the transformation that matches the shape *and* gives an interpretation you can say out loud. A better fit you can't explain is rarely worth it.

Multicollinearity: predictors that say the same thing

- When two predictors are highly correlated (say, square footage and number of rooms), the model can't tell which one deserves the credit.
- The coefficients become **unstable**: large standard errors, wide CIs, and signs that can flip if you nudge the data.
- Prediction is usually fine. It's the *interpretation* of the individual coefficients that falls apart.

Principle: shared credit is unstable credit

Collinear predictors split the credit in an arbitrary way. A big SE on a coefficient is often the model telling you "another variable already explains this."

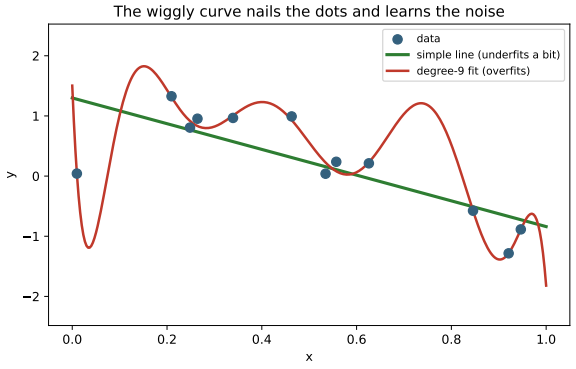
How to recognize it, and what it means

- Warning signs: a predictor that "should" matter looks insignificant, coefficients swing wildly when you add or drop a related variable, or two variables are obviously measuring the same thing.
- For **prediction**, you may not care: the team of predictors still forecasts well.
- For **explanation**, you must care: you can't trust either coefficient alone.

Trap: declaring a variable "unimportant"

A collinear predictor can look insignificant only because its twin is in the model. Drop the twin and it springs back to life. Insignificant is not the same as unimportant.

Overfitting: learning the noise



The wiggly curve passes through almost every point. It has the lowest error on this data.

But it has learned the **noise**, the random wobble that won't repeat. On new data it will do worse than the boring straight line.

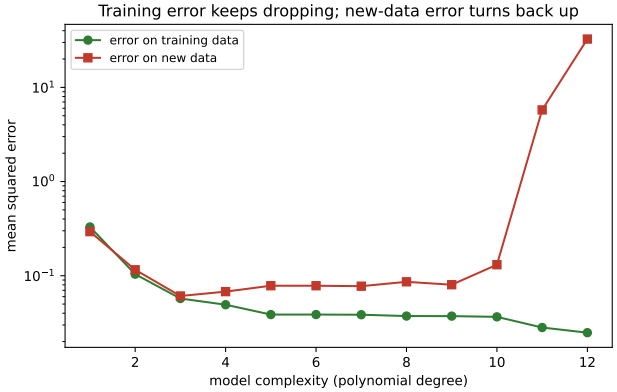
Why R^2 alone will mislead you

- Adding any predictor, even pure random noise, can only make in-sample R^2 go **up** or stay the same. It never goes down.
- So "R^2 improved" is not evidence of a better model. It's almost guaranteed.
- The honest questions are: how does it do on *new* data, and is the gain worth the extra complexity?

Trap: the R^2 chase

Judging a model by training R^2 rewards complexity for its own sake. It's the single most common way people fool themselves with a model.

Training error vs new-data error

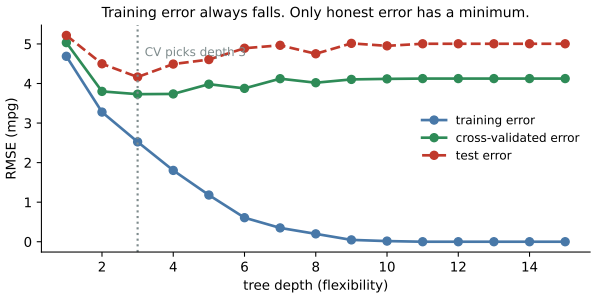


Training error keeps falling as the model gets more complex.

New-data error falls, bottoms out, then climbs as the model starts memorizing noise.

The best model is near the bottom of the new-data curve, not the bottom of the training curve. This is why we held people out in the activity.

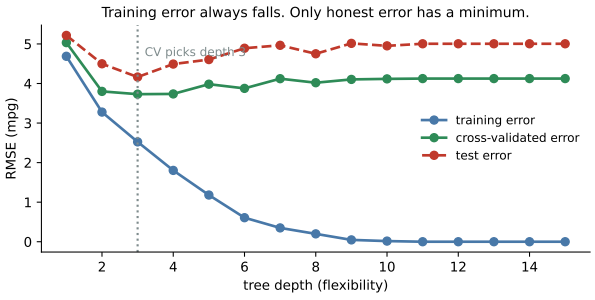
The complexity curve, on real data



120 training cars, the 6 real predictors plus **20 columns of pure noise**, which is the normal condition of a wide dataset.

- Training error goes to zero. The model is perfect on the data it saw.
- Honest error bottoms out at depth 3 and then gets steadily worse.

The complexity curve, on real data



120 training cars, the 6 real predictors plus **20 columns of pure noise**, which is the normal condition of a wide dataset.

- Training error goes to zero. The model is perfect on the data it saw.
- Honest error bottoms out at depth 3 and then gets steadily worse.

The deepest tree is 20% worse than the shallow one, while looking flawless in training.

Two ways to be wrong

Too simple

- Wrong in the same direction every time.
- Fit it on fresh data and you get almost the same wrong answer.
- Stable, and stably off.

(**bias**)

Too complicated

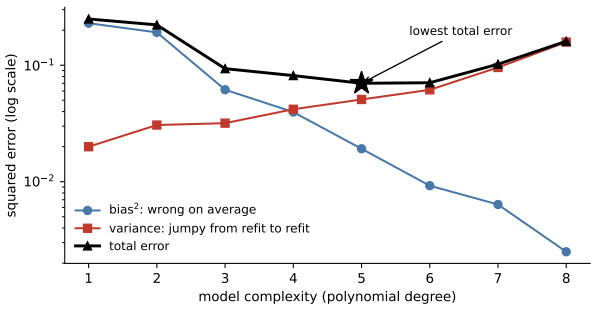
- Right on average across many refits.
- But any single fit chases the noise in the rows you happened to draw.
- Refit on new data and it looks like a different model.

(**variance**)

Principle: why the error curve is U-shaped

Adding flexibility trades one for the other. Bias falls, variance rises, and the total is worst at both ends. The whole art is finding where the trade stops paying.

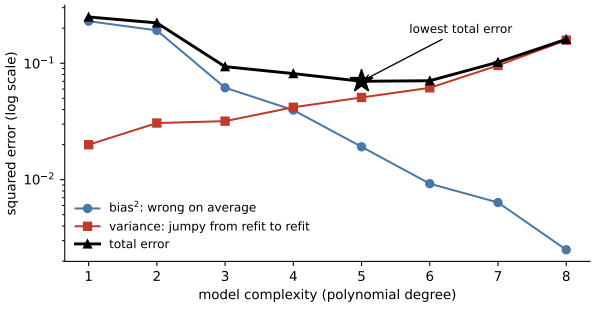
The U-shape, taken apart



600 datasets of $n = 60$, each fit with polynomials of every degree, compared against the true curve we simulated.

- Bias falls the whole way. More flexibility really does get closer to the truth on average.
- Variance climbs the whole way.
- Their sum bottoms out at degree 5.

The U-shape, taken apart

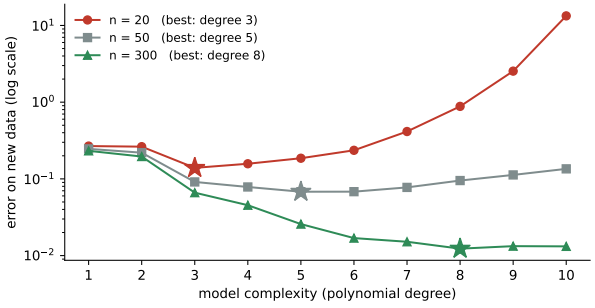


600 datasets of $n = 60$, each fit with polynomials of every degree, compared against the true curve we simulated.

- Bias falls the whole way. More flexibility really does get closer to the truth on average.
- Variance climbs the whole way.
- Their sum bottoms out at degree 5.

Neither curve has a minimum. Only the sum does.

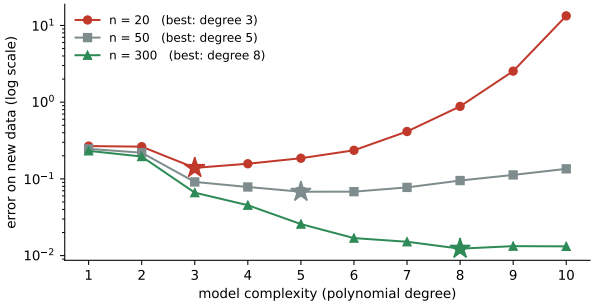
Can you afford the truth?



The true curve never changes. The only thing that changes is how many rows you were given.

- $n = 20$: best is degree **3**.
- $n = 50$: best is degree **5**.
- $n = 300$: best is degree **8**.

Can you afford the truth?



The true curve never changes. The only thing that changes is how many rows you were given.

- $n = 20$: best is degree **3**.
- $n = 50$: best is degree **5**.
- $n = 300$: best is degree **8**.

Overshooting to degree 10 costs you a factor of **96** at $n = 20$, and **7 percent** at $n = 300$. Same mistake, same model, opposite verdict.

What moves the best complexity

When this goes up	Best model gets	Because
Sample size n	more complex	variance is the part data buys down
Noise in the outcome	simpler	more of what you would fit is noise
Candidate predictors p	simpler	searching is itself a form of fitting
Smoothness of the truth	simpler	there is less structure to find
Cost of a confident error	simpler	stable models fail less spectacularly
Need to explain a coefficient	simpler	you cannot defend what you cannot read

Principle: the one-line version

More rows push you toward flexibility. Noise, candidates, and the need to explain push you back.

K-fold cross-validation

1. Split the data into K parts (5 or 10 is standard).
2. Train on $K - 1$ of them, measure error on the one you held out.
3. Rotate through all K, and average the errors.

- Every observation is used for training and, exactly once, for evaluation.
- The result estimates how the *procedure* performs on new data, which is what you actually want to know.

Principle: cross-validation evaluates a recipe, not a model

It tells you how well "fit a depth-3 tree to data like this" works. That is why you refit on all the data once the recipe is chosen.

Doing cross-validation wrong

The most common and most damaging error in applied machine learning:

- Standardizing, imputing missing values, or selecting features using **the whole dataset**, and only then splitting into folds.
- Every one of those steps has now seen the held-out data. The reported error is optimistic, sometimes wildly so.
- The fix: every preprocessing step goes *inside* the fold, applied using only the training portion. This is what a pipeline object is for.

Trap: leakage through the back door

Choosing your 20 best features by correlation with the outcome, and then cross-validating the model on those 20 features, is not cross-validation. The selection already used the answers.

The one-standard-error rule

- Cross-validated errors are themselves estimates, with standard errors.
- If a depth-3 tree and a depth-7 tree are within one standard error of each other, they are not distinguishable, and you should take the **simpler** one.
- Simpler models are more stable over time, easier to explain, cheaper to maintain, and less likely to break under distribution shift.

In practice: a strong thing to say

"The two models were within noise of each other on cross-validation, so I shipped the simpler one and spent the time on data quality instead."

Your turn #2

Your turn: find the leak

A teammate reports 94% cross-validated accuracy on a churn model. Their pipeline:

1. Fill missing values with the column mean, computed over all rows.
2. Keep the 30 features most correlated with churn.
3. Run 5-fold cross-validation on the resulting dataset.

With a neighbor: what is wrong, and is the true accuracy higher or lower than 94%?

Your turn #2: answer

- Steps 1 and 2 both used the *entire* dataset, including every fold's held-out rows. The feature selection is the serious one: it looked at the outcome.
- The true accuracy is **lower**, and with many candidate features it can be far lower.
- Correct order: split first, then impute using training-fold means, then select features using only the training fold, then evaluate.

Principle: the rule that prevents most leakage

Anything that looks at the outcome, or at all the rows at once, must happen inside the fold.

Scenario: the R^2 jumped to 0.95

Setup. A teammate adds 40 features, and training R^2 rises from 0.62 to 0.95 and they want to ship it.

- In-sample R^2 almost always rises with more features, so 0.95 on its own is no evidence at all.
- The test is out-of-sample: split the data, or cross-validate, and compare error on data the models never saw.

You may be asked: how to answer

"R^2 on the training data only ever goes up, so that jump is expected, not impressive. I'd hold out a test set and compare new-data error. If the 40-feature model isn't better there, it's just overfitting."

Why the two error curves diverge

- The error on the 30 cars falls after *every single split*. It has to: more groups means a closer fit to the points being fitted.
- The error on the 10 held-out cars falls for the first few splits, flattens, and then climbs.
- Nobody in the room can feel where that turn happens, and the training error gives no warning at all.

Principle: why the stopping rule cannot come from the training data

You cannot detect overfitting from the data you fit. That is the entire job of the held-out set, and of the cross-validation we set up earlier.

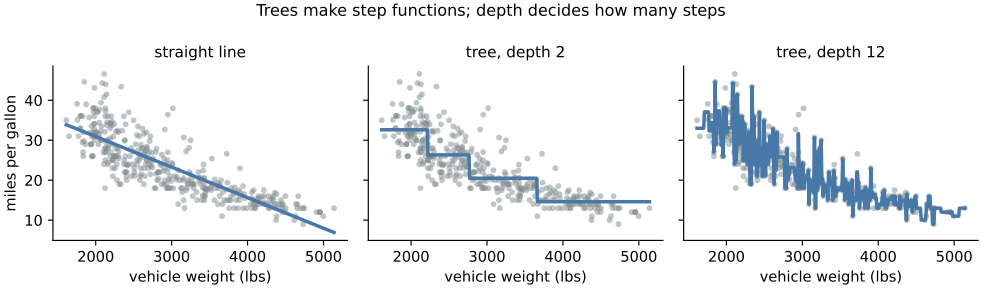
What a tree does

- Repeatedly split the data on one variable at a time, choosing the split that most reduces error within the resulting groups.
- Predict the average outcome (or the majority class) inside each final group.
- The result is a **step function**: no equation, no coefficients, just a sequence of yes/no questions.

Principle: trees are good at what lines are bad at

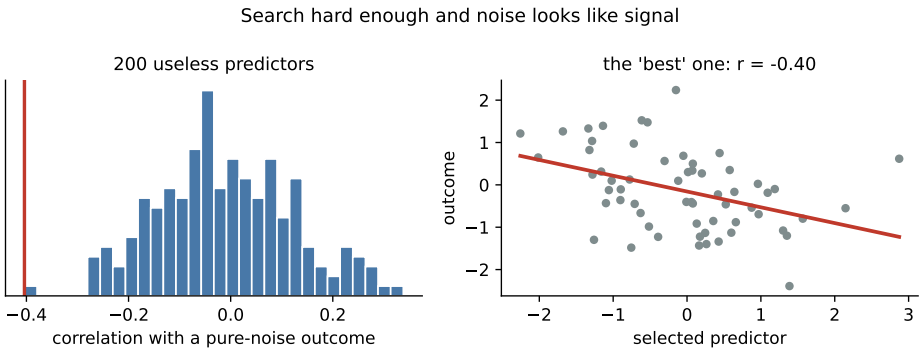
Interactions and sharp thresholds come free, because every split happens inside the region carved out by earlier splits. Smooth trends, on the other hand, take a tree many clumsy steps.

Three fits to the same 392 cars



A line imposes a shape the data does not quite have. A depth-2 tree captures the big picture with four steps. A depth-12 tree traces individual cars, which is not a discovery about fuel economy.

Searching for variables is a form of overfitting



200 predictors, 60 observations, and an outcome that is *pure noise*. The best of the 200 correlates at 0.40 with the outcome and would be significant at $p < 0.01$ if you reported it without mentioning the search.

What that means for stepwise selection

- Automated forward/backward selection by p-value or AIC picks the winners of many comparisons, so the surviving coefficients are biased away from zero.
- The printed p-values and confidence intervals are then **invalid**, because they assume the model was chosen before the data was seen.
- Symptom: an impressive stepwise model that falls apart on next quarter's data.

Trap: significance after selection

If the variables were chosen by looking at the data, do not report their p-values as if they were. Report out-of-sample performance instead, or validate on fresh data.

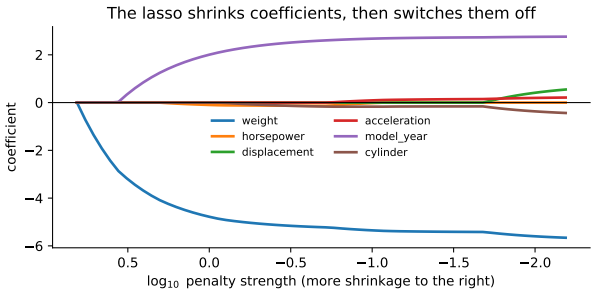
Regularization: shrink instead of choose

- Instead of keeping or dropping variables, penalize large coefficients and let the fit balance accuracy against size.
- **Ridge** shrinks all coefficients toward zero, which is ideal when many predictors each matter a little and are correlated.
- **Lasso** shrinks and sets some exactly to zero, producing a sparse, readable model.
- The penalty strength is chosen by cross-validation, not by taste.

Principle: this is the bias-variance trade you already know

Regularization is the shrinkage from the estimation module, applied to a whole vector of coefficients. A little bias, a lot less variance.

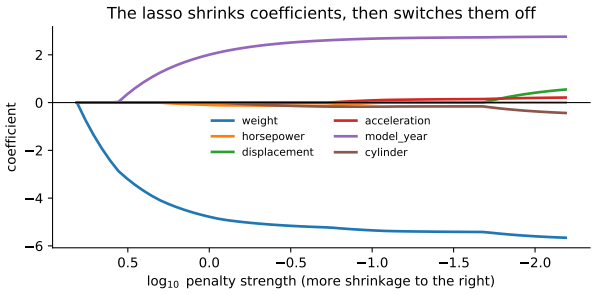
The lasso path, on the cars data



Reading right to left, the penalty weakens and predictors enter the model one by one.

- The order of entry is a rough ranking of usefulness.
- Correlated predictors (weight and displacement) trade places: the lasso picks one and suppresses the other.

The lasso path, on the cars data



Reading right to left, the penalty weakens and predictors enter the model one by one.

- The order of entry is a rough ranking of usefulness.
- Correlated predictors (weight and displacement) trade places: the lasso picks one and suppresses the other.

That last point is a warning: "the lasso dropped it" does not mean "it does not matter."

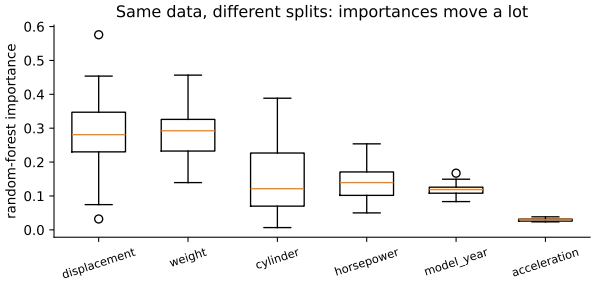
Why one tree is rarely enough

- A single deep tree is **unstable**: change a few rows and the top split can change, rewriting the whole tree.
- **Random forests** average many trees, each fit to a resample of the data and a random subset of features. Averaging kills the variance, exactly as in the pooling picture from the expectation module.
- **Boosting** instead fits trees in sequence, each one correcting the previous errors. It is usually the strongest option on tabular data, and the easiest to overfit.

Principle: ensembles trade explanation for accuracy

You give up the single readable set of rules and get a model that predicts better and can no longer be printed on a slide.

Feature importance is not an effect



Thirty random forests on thirty random splits of the same cars.

- The importance of a given predictor swings substantially from run to run.
- Correlated predictors split the credit arbitrarily between them.

Importance means "this column was useful for splitting," not "changing this would change the outcome."

Your turn #3

Your turn: which model do you ship?

Two models for predicting customer churn:

- Logistic regression: cross-validated AUC 0.79 ± 0.02, six coefficients you can read.
- Gradient boosting: cross-validated AUC 0.81 ± 0.02, 400 trees.

With a neighbor: which do you ship, and what would change your answer?

Your turn #3: answer

- The two are within a standard error of each other, so "more accurate" is not established.
- Ship the **logistic model** if the output must be explained to customers or regulators, if the team maintaining it is small, or if you want to reason about why predictions change.
- Ship the **boosted model** if the decision is fully automated, the gain is real at scale (one AUC point on ten million decisions is money), and you have monitoring in place.

Principle: accuracy is one input to the decision, not the decision

The deployment context, not the leaderboard, chooses the model.

Predict vs. explain: pick one on purpose

Predicting

- You want accurate $\hat{y}$ for new cases.
- You don't need clean coefficients.
- Judge it by out-of-sample error.

Explaining

- You want to understand one effect.
- Coefficients must be trustworthy.
- Worry about confounders and collinearity.

Principle: the goal decides the model

A great prediction model can have meaningless coefficients, and a clean explanatory model can predict poorly. Decide which you need *before* you build, because it changes every choice.

LLM check: mistake or not? (1 of 2)

LLM check: we asked an LLM to validate a model

We told it: "I tried 40 model configurations, picked the one with the best test-set accuracy (91%), and want to report that number. Is that fine?"

It answered: "Yes, that is standard practice. Since you evaluated on a held-out test set rather than the training data, 91% is an unbiased estimate of future performance."

Mistake, or no mistake?

LLM check: mistake or not? (1 of 2)

LLM check: we asked an LLM to validate a model

We told it: "I tried 40 model configurations, picked the one with the best test-set accuracy (91%), and want to report that number. Is that fine?"

It answered: "Yes, that is standard practice. Since you evaluated on a held-out test set rather than the training data, 91% is an unbiased estimate of future performance."

Mistake, or no mistake?

Trap: verdict: mistake

Choosing the best of 40 configurations *by test-set score* makes the test set part of the training process. 91% is the maximum of 40 noisy numbers, so it is biased upward. You need a third split, or nested cross-validation, and you report the score from data used exactly once.

LLM check: mistake or not? (2 of 2)

LLM check: same idea, a different answer

We told it: "Two of my predictors are highly correlated, and both have huge standard errors and look insignificant, yet the model predicts well. What's happening?"

It answered: "That's classic multicollinearity: the two carry overlapping information, so their individual coefficients are unstable with large SEs, even though joint prediction is fine. Decide whether you really need both."

Mistake, or no mistake?

LLM check: mistake or not? (2 of 2)

LLM check: same idea, a different answer

We told it: "Two of my predictors are highly correlated, and both have huge standard errors and look insignificant, yet the model predicts well. What's happening?"

It answered: "That's classic multicollinearity: the two carry overlapping information, so their individual coefficients are unstable with large SEs, even though joint prediction is fine. Decide whether you really need both."

Mistake, or no mistake?

Principle: verdict: no mistake

This is a correct, careful answer. It separates unstable *coefficients* from fine *prediction*, which is exactly the right distinction.

The arc of a modeling project

1. **Split first.** Hold out a test set and do not touch it again until the end.
2. **Start with the simple model.** It is your benchmark, and often your answer.
3. **Build the whole pipeline inside the folds**, preprocessing included.
4. **Tune complexity by cross-validation**, and apply the one-standard-error rule.
5. **Compare candidates on the same folds**, not on separately quoted numbers.
6. **Touch the test set once**, to report a final number.
7. **Explain what the model uses**, and say plainly that importance is not causation.

Building a model you can defend

1. **Split first.** Hold out a test set and do not touch it again until the end.
2. **Fit the simple model** as your benchmark, and always compare against predicting the mean.
3. **Put every preprocessing step inside a pipeline** so it is refit within each fold.
4. **Cross-validate:** `cross_val_score(pipe, X, y, cv=KFold(5, shuffle=True))`.
5. **Tune complexity on the CV curve**, and take the simplest model within one standard error of the best.
6. **Compare candidates on the same folds**, and look at the fold-to-fold spread before declaring a winner.
7. **Touch the test set once** and report that number.

The traps, all in one place

Trap: the ones that bite most often

- Judging a model on training error, or on R^2 alone.
- Preprocessing, imputing, or selecting features outside the folds.
- Choosing among many models by test-set score and then reporting that score.
- Quoting p-values from a model whose variables were chosen by search.
- Reading feature importance as a causal effect.
- Preferring the complex model when the simple one is within noise of it.
- Interpreting individual coefficients of two nearly identical predictors.

What you should be able to do with software

Nobody is asking you to memorize syntax. You are expected to know *which* procedure the question calls for, what to hand it, and what the output means. With an AI assistant open, you should be able to produce any of these and defend the result.

You should be able to	**What you would reach for**
Train/test split	`train_test_split`
Cross-validation on the same folds	`KFold(5, shuffle=True, random_state=...)`
Preprocessing that cannot leak	`make_pipeline(StandardScaler(), model)`
Ridge and lasso with the penalty chosen for you	`RidgeCV`, `LassoCV`
A tree, and a forest	`DecisionTreeRegressor`, `RandomForestRegressor`
Feature importance, with its caveats	`model.feature_importances_`
The one-standard-error rule	CV mean plus `scores.std()/sqrt(k)`

If you cannot say what the output means, the fact that you produced it is worth nothing.

Where we go next

- We now have models we can trust, judged on data they have never seen.
- **Unit 5** zooms all the way in on a single **prediction**: when the model hands you one number for one case, how wrong is it likely to be, and where does that error come from?

Principle: the habit to keep

Flexibility is not the goal. Honesty about performance is. A simple model you can check beats a complex one you cannot.